

FakeMedia on MultiChain

Architecture, workflow, evidence and reproducibility

Preserved implementation: 2.0.2-mining-check
MultiChain Community 2.3.3 | Python 3.10+ | Five local processes

How to use this companion

Use this reference to understand the code, prepare the live demonstration and write your own implementation report. It is AI-assisted supporting material, not the student-authored report required by the assessment. The editable workbook is supplied separately.

What is established

Evidence	Supported scope
Provided Windows console log [E1]	Successful five-node readiness and native authority CLI queries.
Local package tests [E2]	99 Python tests passed. No native MultiChain daemon ran during those tests.
Application/security/load execution	Still requires the actual demo, security and benchmark output from the student machine.

The code is a paper-inspired adaptation. It does not reproduce the original Fabric/Composer deployment, 900,000-record dataset or deep reinforcement-learning training. See the mapping and limitations before making any claims.

Reference keys [1]-[8], [C], [E1] and [E2] are defined on the final page and in REFERENCES.md. Figures describe architecture or intended code behavior; they are not execution screenshots. No marks or AI-writing percentage are certified.

1. Research problem and selected paper

Problem and proposed solution [1]

Shahbazi and Byun address the lack of trustworthy social-media news and user accounts. Their proposed system combines natural language processing, reinforcement learning and a permissioned blockchain. The intended workflow analyzes news, authenticates participants and records transaction history in a distributed ledger.

Section III.A describes cleaning, segmentation, stop-word handling, feature extraction, indexing and embeddings. Section III.B describes deep reinforcement learning and a Markov decision-process formulation. Section III.C identifies the user, publisher, validator, transaction and news entities. The paper uses credibility information and a PoA-oriented design for authorization and validation.

Original implementation [1]

Section III.C.2 describes Hyperledger Composer business logic. Section V and Table 6 report Hyperledger Fabric 1.2, Composer Playground/REST tooling, Node.js, CouchDB, Docker and Ubuntu. The paper also reports a data collection of 900,000 news records from 2015-2019. Those records and the authors' exact model implementation are not included in this teaching package.

Consensus terminology: preserve the distinction

The design and conclusion discuss PoA, while Section VI.B discusses PBFT and RAFT evaluation. These are the paper's separate descriptions. The supplied text does not establish one complete, reproducible deployment configuration joining all of them; this adaptation does not silently replace that uncertainty with an invented consensus setup.

Conclusion direction addressed

The conclusion identifies further investigation of the Proof-of-Authority protocol and user validation. The adaptation addresses a bounded user-validation/authority-management direction through signed role enrollment, restricted stream writers and revocation. It does not claim to solve all fake-news or identity problems. [1, Section VII; C]

2. Original architecture and data flow

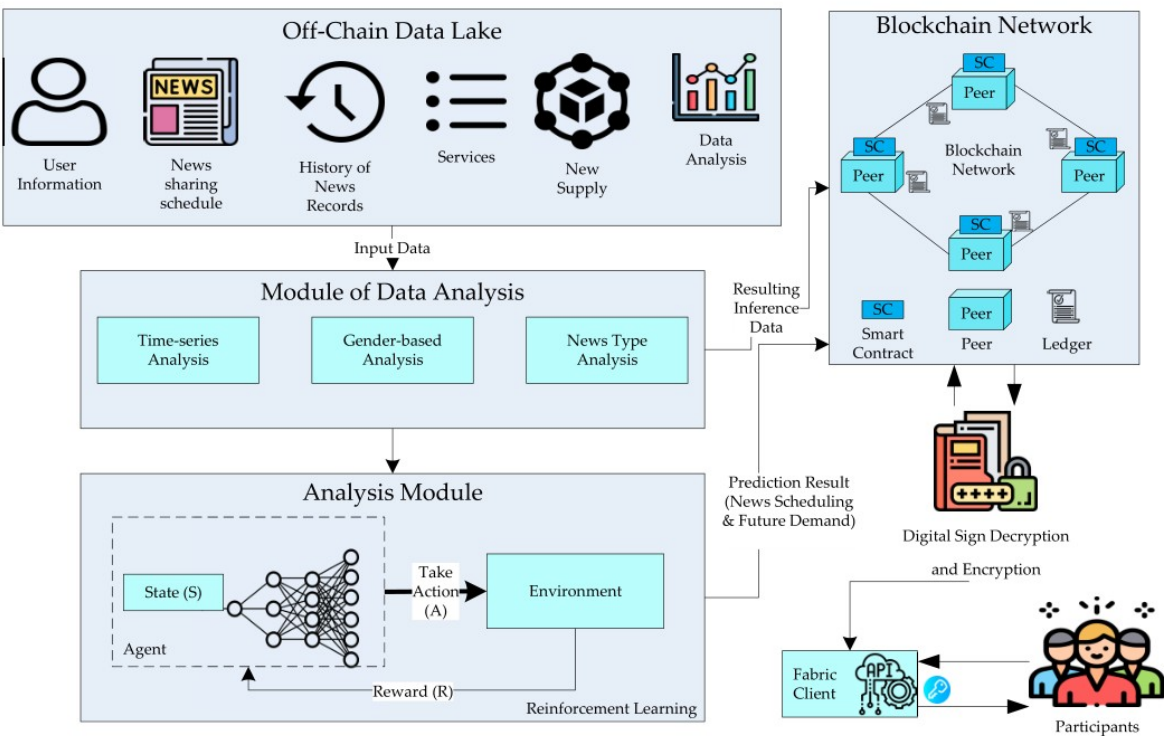


Figure F01. Original Figure 1, cropped from Shahbazi and Byun [1], PDF p.3 / printed p.128444. CC BY 4.0. This is the paper authors' figure.

Figure F01 links the off-chain data lake to data-analysis and reinforcement-learning modules. The analysis outputs and participant interactions connect to a blockchain network containing peers, smart contracts and the ledger. Participant access is shown through Fabric-client/API and digital-sign/encryption components. [1, Figure 1]

Original element	Explanation from the paper
Data lake / analysis	User/news information, history and feature-oriented analysis feed prediction.
Participants / clients	Participants interact with the permissioned blockchain through the client interface.
Blockchain / smart contract	Business rules manage records, access and transaction history.
Reviewer entities	Publishers submit news; validators examine it; transactions record acceptance/rejection.

The original figure is conceptual: it should not be used as evidence that this MultiChain lab deploys Fabric clients, Composer or the authors' DRL architecture.

3. Component mapping and justification

Paper component [1]	MultiChain implementation [C]	Boundary
User identification	Authority approval + signed ENROLL in registry	Key possession, not real-world identity.
Publisher	Dedicated wallet; news.write	Cannot normally write decisions.
Validators	Two configured wallets; votes.write	Fixture reviewers are not independent fact checkers.
News / transaction history	JSON streams, content hash, txids and confirmations	Full article text is visible to chain participants.
NLP / DRL	TF-IDF evidence and terminal tabular-Q model	Teaching substitute; not original model or dataset.
Smart contract logic	Python submit/vote/finalize/inspect; native ACLs	Not on-chain Fabric chaincode; raw RPC can bypass wrapper rules.
PoA-oriented trust	Three native mining addresses; diversity 0.6	MultiChain mining, not a PBFT/RAFT reproduction.
Credibility / incentives	Descriptive decision score; NewsCredit asset	Neither automatically controls permissions.
Queries / audit	Stream indexes and auditor recomputation	Bounded evidence checking, not factual truth.

The key mapping is functional, not platform equivalence. The application decides how to interpret review records; the blockchain decides which signed, permitted transactions can be recorded. [C]

The full mapping, source locations and omissions are in docs/PAPER_MAPPING.md. Explain the justification for each chosen mapping in your own report; do not present the table as proof of research-model equivalence.

4. Proposed MultiChain architecture

PROPOSED MULTICHAIN CLASSROOM ARCHITECTURE
Five independent wallets / processes, one shared chain

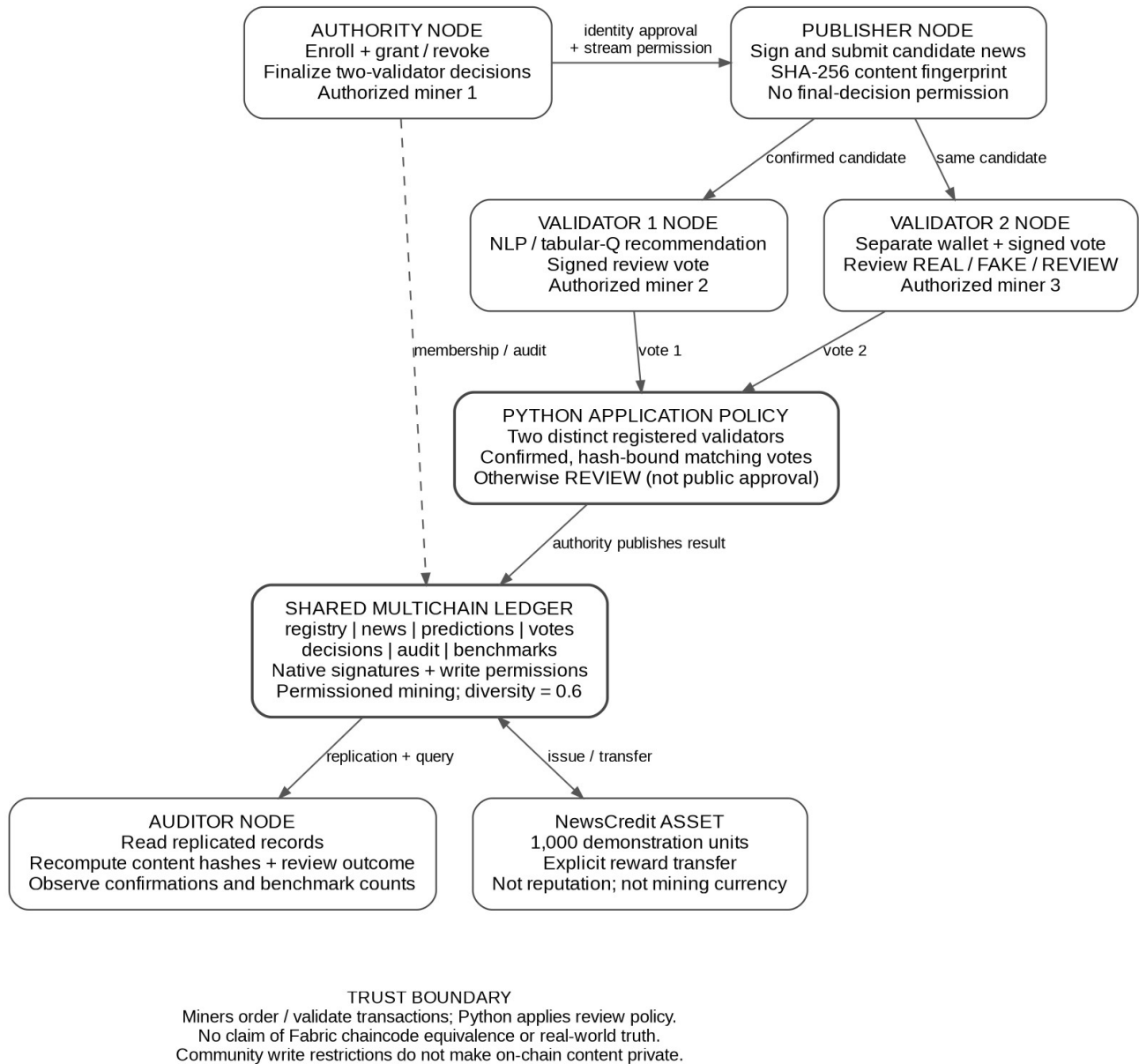


Figure F02. Paper-inspired classroom architecture for the preserved implementation [C]. Separate wallet roles share a replicated ledger; the Python policy remains outside native consensus.

5. Nodes, processes and ports

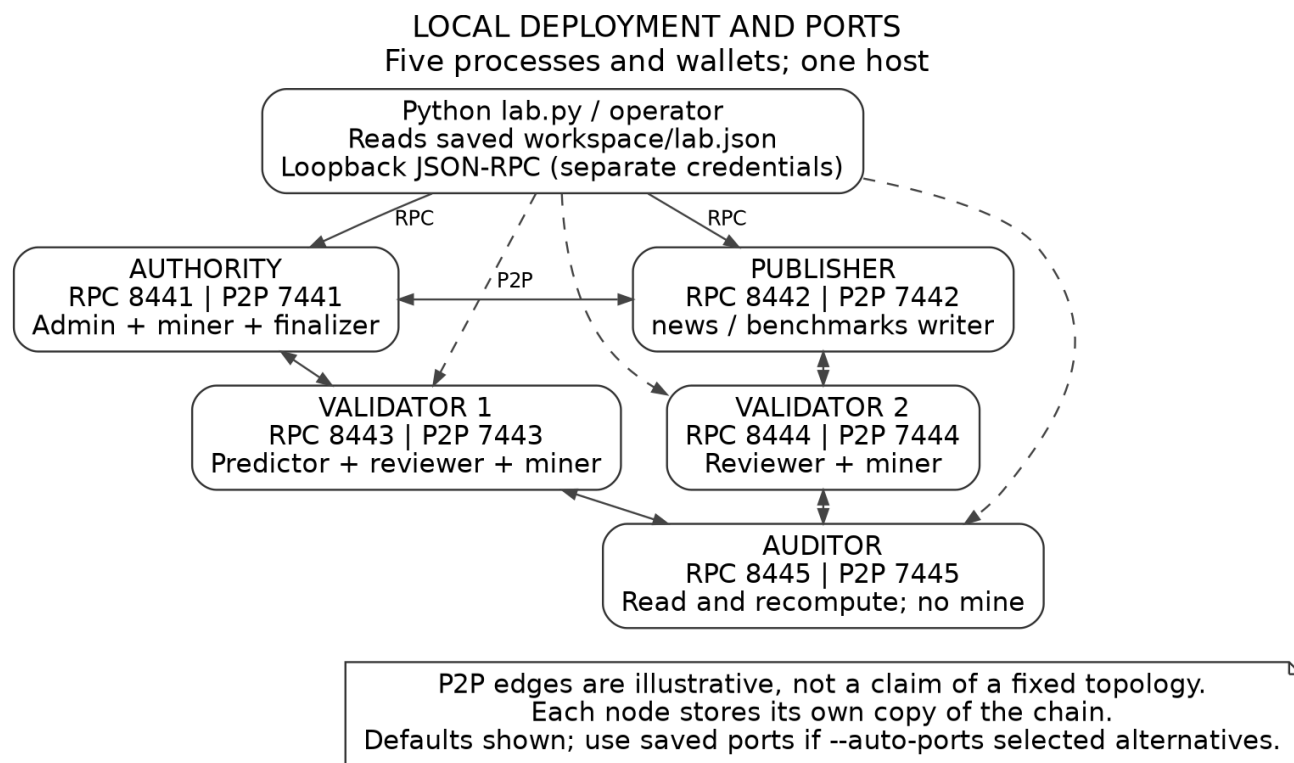


Figure F03. Default local topology [C]. Edges are illustrative. Use actual saved ports when --auto-ports selected alternatives.

The roles are separate processes and wallets on one host. All are controlled by the local operator; they are not five independent organizations. Each node has a separate private data directory and RPC configuration.

RPC ports 8441-8445 serve local application/CLI calls; P2P ports 7441-7445 serve node connections. Protocol version 20013 is neither a node address nor a port. A seed needs host:P2P-port. [C; 2]

The core wrapper reads the selected workspace rather than relying on the shell's current directory. A bare multichaind command without the matching data directory may initialize or access an unrelated default-directory node. Keep the existing workspace and use start for a completed lab. [C; 2]

6. Configuration and genesis

Requested parameter	Default	Purpose / qualification
target-block-time	2 s	Target average; not a latency guarantee.
maximum-block-size	8,388,608	8 MiB maximum block size.
mining-diversity	0.6	Restricts repeated mining by recent miners.
mining-turnover	0	Low-turnover setting for this local lab.
mine-empty-rounds	-1	Keep blocks available for confirmation observations.
setup-first-blocks	10	Benchmark only after all nodes pass setup phase.
root-stream-open	false	Root writes are restricted.
anyone-can-* selected flags	false	Explicit connect/send/receive/create/issue/mine/admin grants.

These are requested defaults from `setup()`, not a substitute for captured runtime evidence. Use `getblockchainparams` to record actual values. The parameter example file is not a complete `params.dat` and must not overwrite an existing chain. [C; 3]

Creation-time versus runtime settings

Chain parameters are supplied before genesis using `multichain-util` overrides. Runtime configuration sets per-node RPC credentials, ports and loopback binding, plus `txindex=1`, `rpcthreads=16` and automatic stream/asset subscription. The live configuration stays private. [C; 2, 3]

Mining authorization versus waiting

The v2.0.2 checker uses `listminers.permitted` for effective authorization. `diversitywaitblocks` is a separate wait. An authorized miner waiting its turn is not an unpermissioned node. Do not disable diversity or grant extra administration simply to obtain a green screenshot. [C; 4]

7. Complete setup / restart workflow

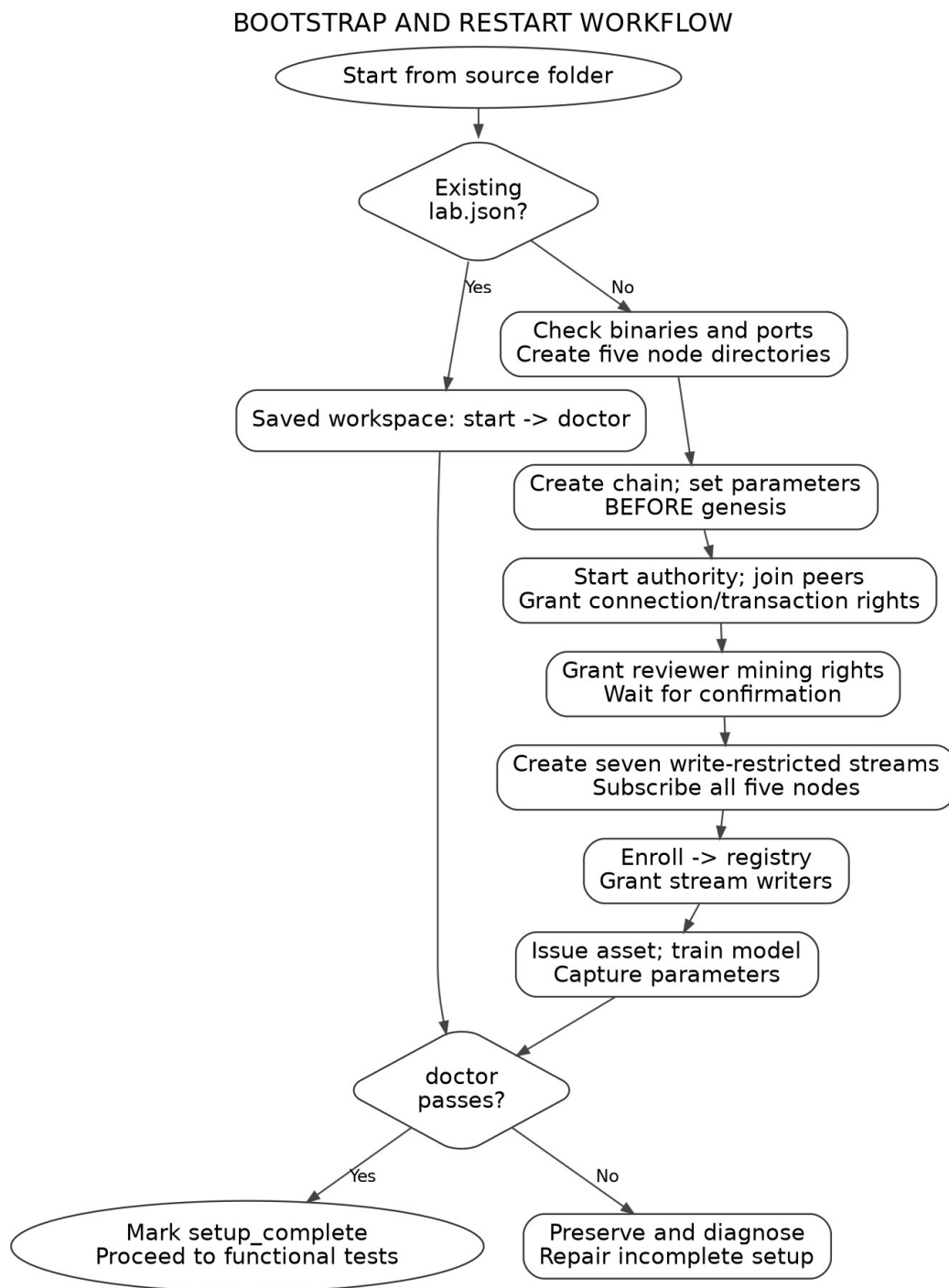


Figure F04. Code-derived bootstrap and reuse flow [C]. start reuses existing state; setup creates new state; repair resumes incomplete setup without deleting wallets.

Read the actual step output and stop on failure. A green readiness check is the entry condition for functional tests, not a throughput result.

8. User validation and enrollment

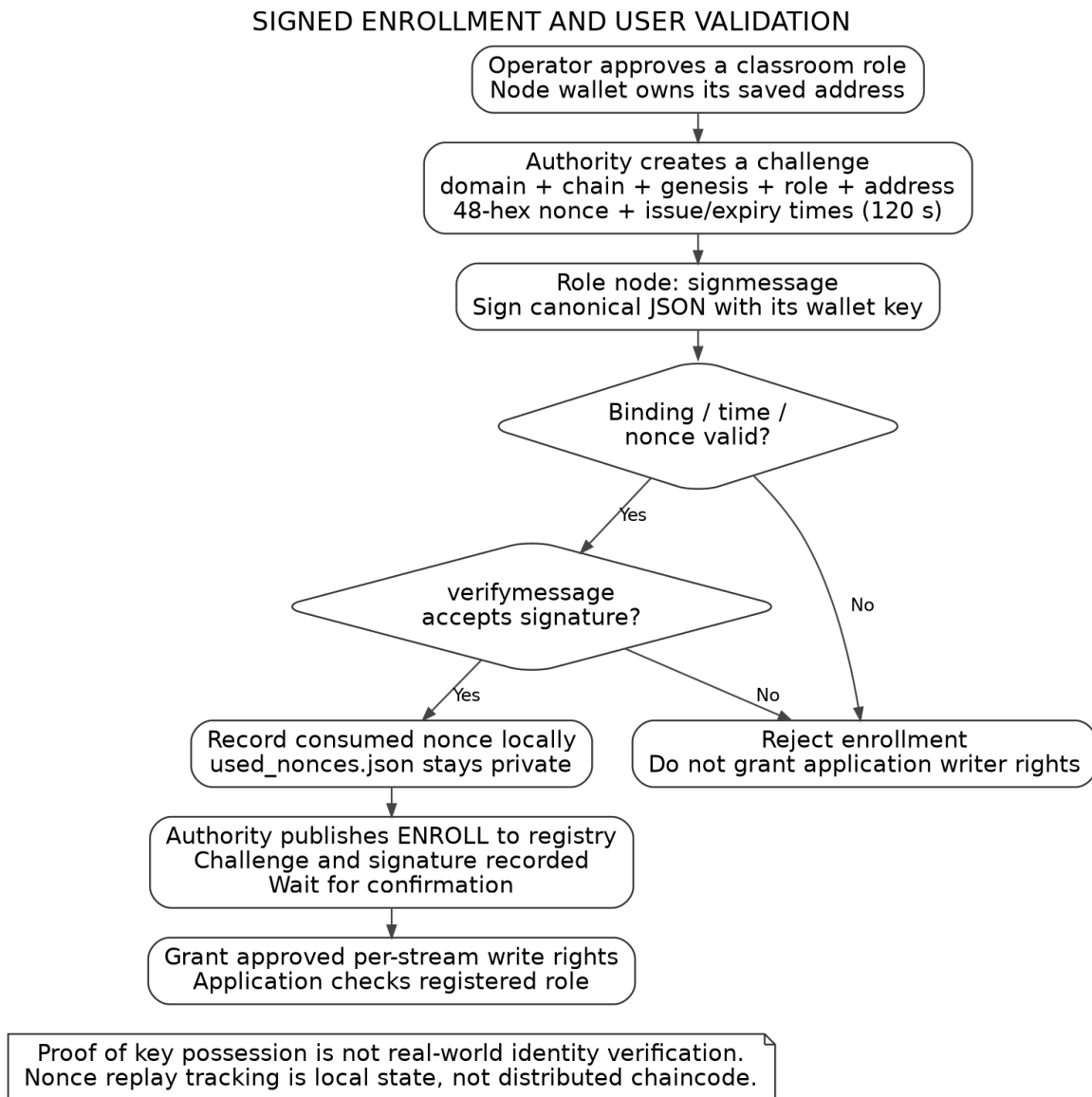


Figure F05. Signed enrollment [C]. The challenge binds the wallet, role and chain instance, while consumed nonces remain local private state.

Security tests exercise key possession, replay, wrong-key signing, role substitution and expiry. Enrollment records are signed publications by the authority. The gateway checks confirmed registration and the chain enforces granted stream rights. [C]

This is the bounded extension connected to the paper's conclusion. Operator identity approval and local nonce storage remain trust assumptions; it is not decentralized KYC or an anonymity protocol. [1, Section VII; C]

9. Article-to-decision implementation

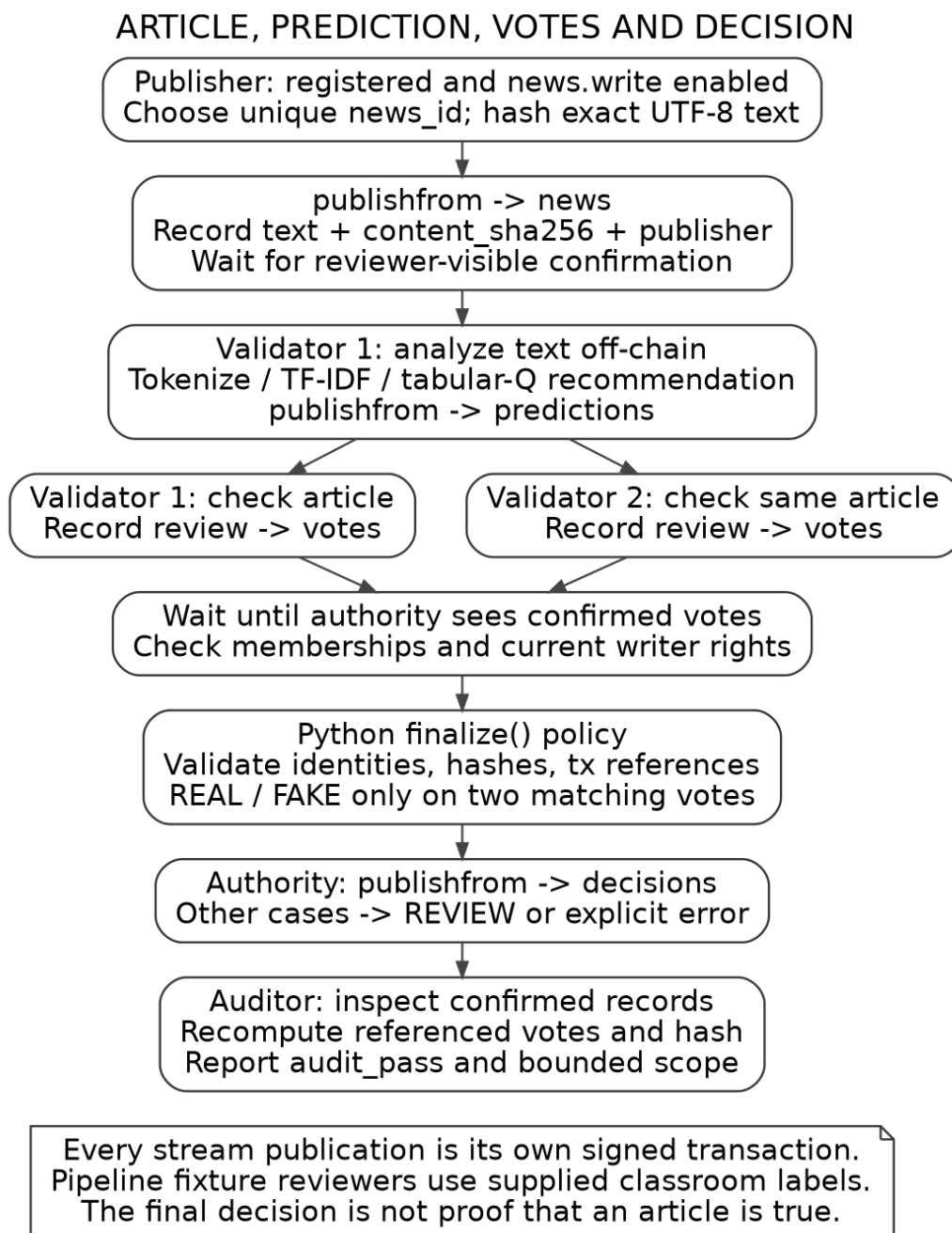


Figure F06. Five-publication workflow [C]: news, prediction, reviewer 1 vote, reviewer 2 vote and authority decision. Auditor inspection is a read/recompute step.

The text hash binds reviews to content; the news txid binds them to the specific publication. Predictions are recommendations. Fixture-mode votes follow classroom labels, not independent evidence gathering.

10. Decision logic and audit boundaries

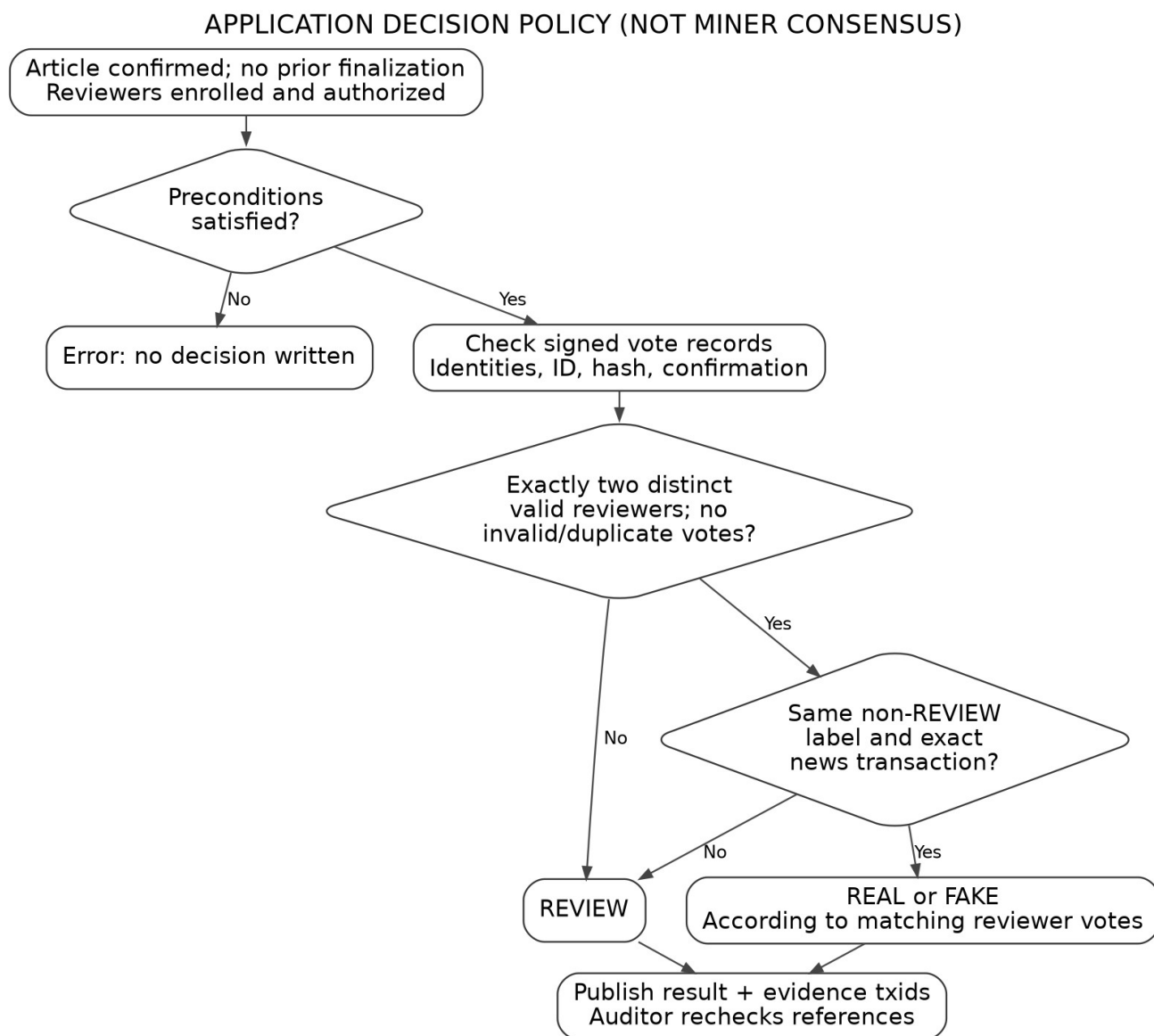


Figure F07. Native miner consensus does not run this Python decision policy [C]. Preconditions can raise errors; insufficient or conflicting review evidence produces REVIEW.

The auditor validates the confirmed article and decision publishers, content hash, referenced vote IDs, article transaction binding and recomputed label. Its audit does not reconstruct all historical permissions or guarantee completeness of all possible conflicting evidence. [C, Lab.inspect]

Separate these claims in the report: transaction recorded, reference integrity verified, reviewer agreement observed, and factual truth established. The first three do not imply the fourth.

11. Teaching model, records and asset

Off-chain NLP and learning [C]

The included CSV contains 32 classroom records: 24 training and 8 test rows. The tokenizer handles English alphabetic tokens and preserves negation. TF-IDF vectors provide cosine evidence. A lexical-count state indexes a terminal Q-learning policy with gamma=0, alpha=0.1 and epsilon=0.2; the default training uses 250 epochs and seed 42. This is a contextual-bandit-style simplification, not the paper's deep RL.

If maximum similarity is below 0.35, the recommendation becomes REVIEW. Similarity is not calibrated truth probability. Model evaluation reports coverage and accuracy on this tiny fixture, not research accuracy or a successful reproduction of the paper. [C]

Record	Key fields and purpose
news	news_id, text, content_sha256, publisher_address, created_at, run_id
predictions	news_id, news_txid, content hash, model hash, label, similarities and threshold
votes	validator_address, news_id, news_txid, content hash, label and reason
decisions	policy name, article reference, content hash, label, reasons and vote txids

Asset and credibility are different

NewsCredit is issued once as 1,000 indivisible demonstration units. reward explicitly transfers units; it does not award native mining currency. reputation derives clip(50 + 5*REAL - 10*FAKE, 0, 100) from authority-issued decisions. This is a teaching score, not the paper's formula, and it does not automatically change mining or publishing permissions. [C]

12. Execution and evidence collection

Run commands from the new source folder, while pointing `--workspace` at the existing private workspace. All source/test files are included. No network reset is necessary. The full Windows paths and self-contained command blocks are in README.md and docs/WORKFLOW.md.

```
python -m unittest -v test_lab test_recovery test_mining_permissions test_submission_tools
python lab.py --version
```

Existing workspace: sequence

Set this to the actual saved workspace before using the commands.

```
$W = Read-Host "Enter the existing workspace directory"
```

```
python .\lab.py --workspace "$W" start
```

```
python .\lab.py --workspace "$W" doctor
```

Continue only after readiness passes.

```
python .\lab.py --workspace "$W" demo
```

```
python .\lab.py --workspace "$W" security-tests
```

```
python .\lab.py --workspace "$W" benchmark --n 10 --workers 2
```

Continue only after the small test passes.

```
python .\lab.py --workspace "$W" benchmark --n 1000 --workers 10
```

```
python .\lab.py --workspace "$W" capture
```

These lines describe the order. Use the fail-fast collector for an automated run that stops on an error. A new PowerShell window does not retain `$W` from a previous one.

Collector and publication-safe export

```
$W = Read-Host "Enter the existing workspace directory"
```

```
python .\tools\collect_evidence.py --workspace "$W" --run-live-tests
```

Run the exporter only after reviewing the collection result.

```
python .\tools\export_evidence.py --workspace "$W"
```

The collector runs real fixture writes and temporary permission changes only with `--run-live-tests`. The exporter uses an allowlist, excludes wallets/configuration, redacts local paths and records original/exported hashes. It cannot guarantee that arbitrary news text or manually added screenshots contain no private information.

13. Security tests and expected outcomes

SECURITY TEST EXECUTION AND RESTORATION

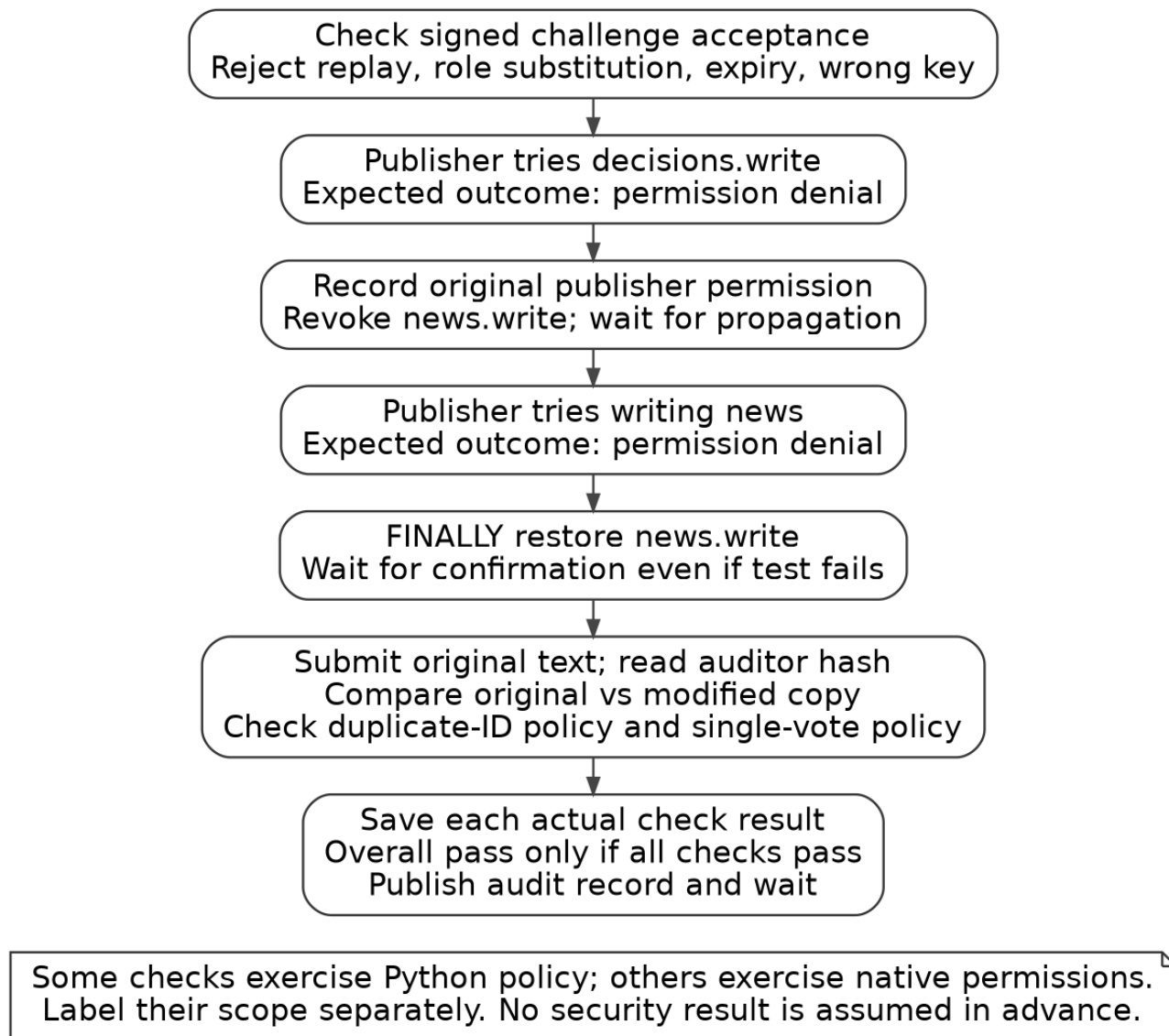


Figure F09. Intended live security-test sequence [C]. Some assertions test the Python policy; others test native stream permissions. No live pass is assumed.

In negative tests, the intended permission/signature rejection is success; a timeout or unrelated error is not equivalent. The publisher's original write permission must be restored and verified before another workload. Keep the real per-test result and audit transaction ID.

14. Burst-load workflow

BURST LOAD AND CONFIRMATION MEASUREMENT

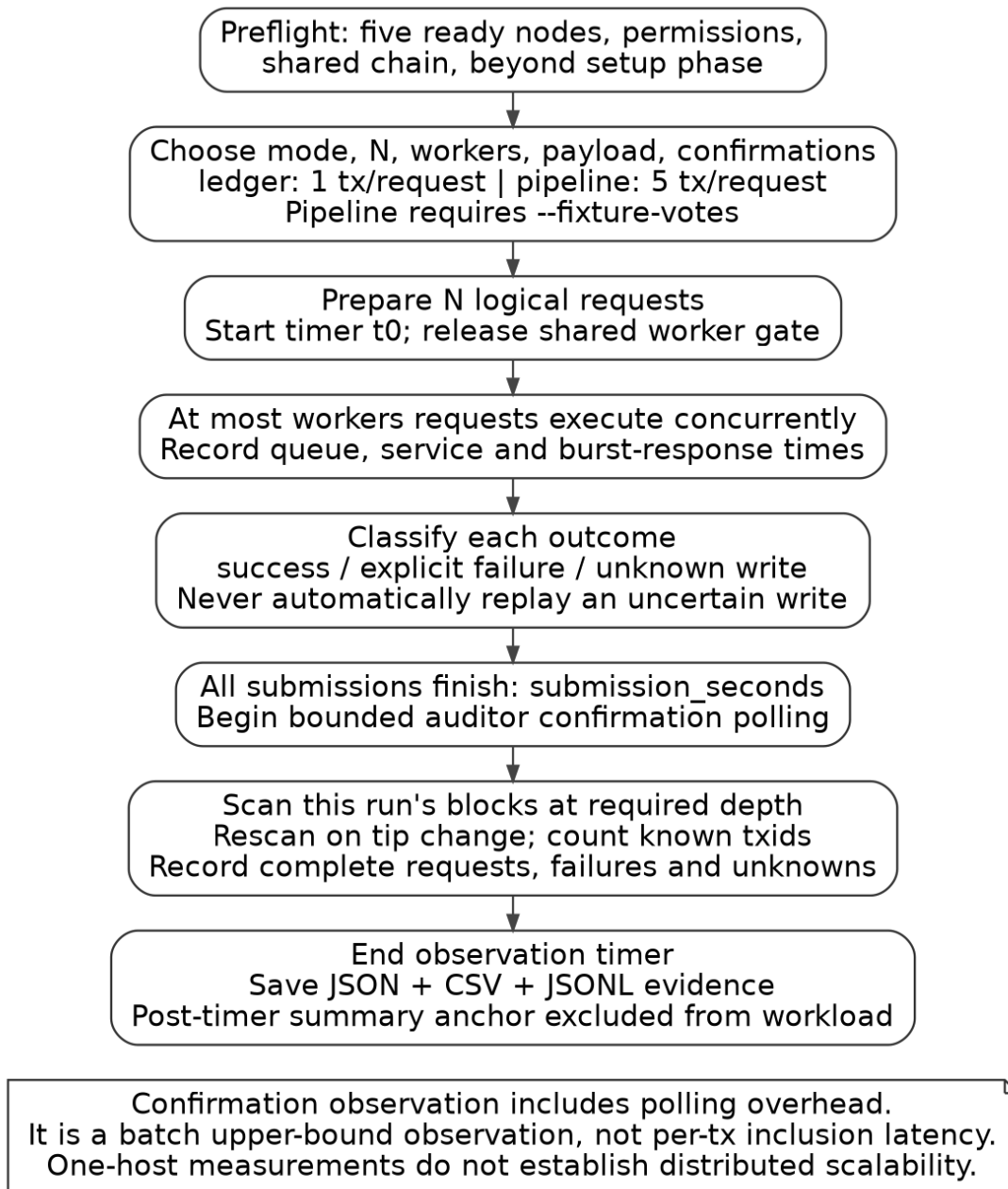


Figure F08. Code-derived measurement flow [C]. N logical requests are released together, at most W workers execute at a time, and the auditor observes confirmations after submission.

Ledger mode is one publication per request. Pipeline mode requires --fixture-votes and is five publications per completed article. Do not compare the two workloads as equal numbers of transactions.

15. Interpreting benchmark results

Field	Interpretation in the preserved code
successes / failures / unknown_outcomes	Keep all three counts. An uncertain write is not silently resent.
known_broadcast_transactions	Distinct transaction IDs returned to the client, including any known partial pipeline work.
confirmed_complete_requests	Successful requests whose known publications satisfy observed confirmation depth.
submission_seconds	t0 until all request attempts complete.
confirmation_observation_seconds	t0 until post-submission confirmation observation ends; includes polling overhead.
service p95 / p99	Successful requests only; service excludes initial worker-queue delay.
burst_response latency	Includes initial queue delay from shared arrival t0.
timeout	Post-submission observer limit, not an overall application runtime limit.

Rates are reported separately for successful requests, returned transactions, confirmed complete requests and confirmed transactions. Record units and the denominator. A failed 1,000-request run finishing quickly is not evidence of high successful throughput. [C]

Change one variable at a time: N, workers, payload or confirmation target. Start with 10 requests, then 1,000; repeat comparisons when time permits. Preserve failures and unknowns. A one-host test does not establish distributed scalability, and its results are not comparable to the paper's platform without a matched experimental design. [C; 1]

Use actual run IDs with reconcile. In pipeline mode add --stream news; this reconciles article submissions, not completed five-stage workflows. Hashes anchored by the application refer to original summary bytes, not redacted/reformatted exported copies. [C]

16. Results supported by the supplied log

Evidence E1 is the student-provided console log captured at 2026-10-08T19:41:12.916+00:00. The selected JSON is preserved in evidence/provided-run with only the workspace path redacted. It is log evidence, not an original screenshot or an independent execution by the package author.

Observed item	Recorded value / bounded claim
Implementation / platform	2.0.2-mining-check / MultiChain 2.3.3
Readiness	setup_complete=true; all_passed=true; issues=[]
Node roles	All five online; intended mining permissions present
Agreement	Shared block 552 hash matched; doctor height_lag=1
Authority query	getinfo reports block 554, 4 connections, 1 asset, 8 total streams, errors empty
Decision records	3 confirmed records in decisions; count alone does not prove valid outcomes
Benchmark stream	0 items at this snapshot; no successful benchmark measurement established

Evidence still to collect

A passing demo summary with per-case audit checks; actual security-test output; the small and large benchmark summaries/traces; real command screenshots; and the published repository URL/commit. Neither typeset logs nor expected-output examples replace those deliverables.

Local package verification [E2]

99 local Python tests passed during packaging: 88 preserved application/regression tests and 11 export/publication-tool tests. No native MultiChain node ran in that local test suite. The application source is byte-identical to the v2.0.2 file used for this package, with a SHA-256 recorded in PACKAGE_PROVENANCE.json.

17. Report completion and submission

Student report

Use the editable workbook to write an abstract, introduction, paper summary, methodology, mapping, step-by-step MultiChain workflow, results, discussion, limitations and conclusion. Insert actual screenshots and label every selected figure and table. Explain the specific observed fields rather than using a screenshot as decoration.

The teacher's AI-writing limit requires predominantly student-authored prose. Generated reference paragraphs and captions are not independent student work. Write from your own notes, record assistance and confirm how the instructor counts code, figures and captions. The package cannot certify an AI-writing percentage or a grade.

Repository

Publish this new source directory, not the old private workspace. README gives clean-run and existing-workspace paths. The repository includes dependencies, tests, figures, a source index and evidence tools. Use the publication guard on staged Git blobs, then manually inspect PDFs and screenshots. Never commit wallets, live RPC configurations, credentials or node data.

Final three deliverables

Deliverable	Completion action
Selected paper PDF	Use paper/Selected_Paper.pdf; preserve attribution.
Implementation report PDF	Replace workbook prompts with your writing and measured evidence; export as submission/Implementation_Report.pdf.
GitHub URL	Create and push the real repository; confirm examiner access; include commit identifier in the report.

The supplied rubric assigns 7.5 marks to this implementation section. No fabricated sub-weights or claim of completing the other practical section are made. Use SUBMISSION_CHECKLIST.md as a completion check, not a mark guarantee.

References and source boundaries

[1] Selected paper

Z. Shahbazi and Y.-C. Byun, Fake Media Detection Based on Natural Language Processing and Blockchain Approaches, IEEE Access 9, 128442-128453, 2021. DOI: 10.1109/ACCESS.2021.3112607. Included PDF; Figure 1 p.3; methodology pp.4-6; implementation p.8; evaluation pp.10-11; conclusion p.11.

[2] Creating / connecting

<https://www.multichain.com/developers/creating-connecting/>

[3] Blockchain parameters

<https://www.multichain.com/developers/blockchain-parameters/>

[4] JSON-RPC API

<https://www.multichain.com/developers/json-rpc-api/>

[5] Permissions

<https://www.multichain.com/developers/permissions-management/>

[6] Streams

<https://www.multichain.com/developers/data-streams/>

[7] GitHub source import

GitHub Docs, Adding locally hosted code to GitHub. Full URL in REFERENCES.md.

[8] GitHub Python CI

<https://docs.github.com/en/actions/tutorials/build-and-test-code/python>

[C] This implementation

lab.py 2.0.2-mining-check; demo_news.csv; code symbol/line index in docs/CODE_INDEX.md. Source for lab-specific behavior, not original-paper claims.

[E1] Student-provided log

Pasted text(3).txt; selected JSON and provenance under evidence/provided-run/.

[E2] Local tests

validation/unit-tests.txt: 99 tests, no live blockchain execution.

Public references checked 2026-10-09. The original paper figure is reproduced as a crop with attribution under its CC BY 4.0 notice. Other figures are AI-assisted code-derived illustrations, not execution evidence.